

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

24 - Arithmetic for Computers: Addition and Subtraction

Introduction

- Computer words are represented as binary numbers
- What about other numbers that commonly occur?
 - Fractions and other real numbers
- What happens if an operation creates a number bigger than can be represented?
- How does HW multiply or divide numbers?
- Representation of real numbers, arithmetic algorithms, HW that follows these algorithms

Addition

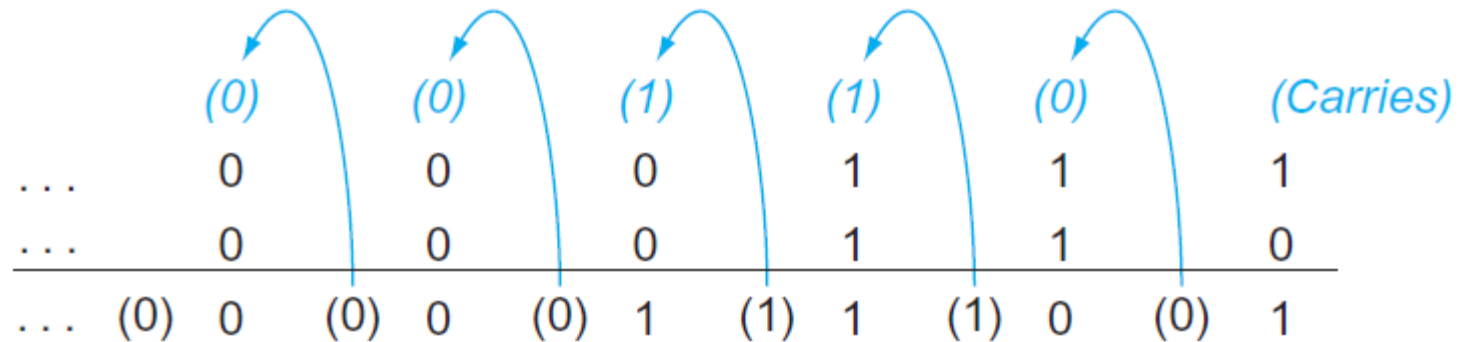
- Digits are added bit by bit RTL
- With carries passed to next digit to left

Example

- Add 6_{ten} to 7_{ten} in binary.

$$\begin{array}{r}
 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{\text{two}} = 7_{\text{ten}} \\
 + \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_{\text{two}} = 6_{\text{ten}} \\
 \hline
 = \quad 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1101_{\text{two}} = 13_{\text{ten}}
 \end{array}$$

- Carries in parentheses, arrows showing how they are passed



Subtraction

- Can be done directly or via addition
- Subtraction uses addition: appropriate operand is simply negated before being added

Example

- Subtract 6_{ten} from 7_{ten} in binary.
- Subtraction can be done directly:

$$\begin{array}{r} \quad 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0111_{\text{two}} = 7_{\text{ten}} \\ - \quad 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0110_{\text{two}} = 6_{\text{ten}} \\ \hline = \quad 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0001_{\text{two}} = 1_{\text{ten}} \end{array}$$

- Or via addition using two's complement representation of 6:

$$\begin{array}{r} \quad 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0111_{\text{two}} = 7_{\text{ten}} \\ + \quad 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1010_{\text{two}} = -6_{\text{ten}} \\ \hline = \quad 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0001_{\text{two}} = 1_{\text{ten}} \end{array}$$

Overflow

- Occurs when result from an operation cannot be represented with the available HW, in this case a 32-bit word
- Cannot occur when adding operands with different signs
 - Sum must be no larger than one of the operands
 - For example, $-10 + 4 = 6$
 - Operands fit in 32 bits; sum is no larger than an operand, it must fit in 32 bits
- No overflow can occur when adding positive and negative operands

Overflow

- In subtraction, when signs of operands are *same*, overflow cannot occur
 - For example, $c - a = c + (-a)$
 - Subtract by negating the second operand and then add
- When subtract operands of same sign, we end up by *adding* operands of *different* signs
- Overflow cannot occur in this case

Overflow Detection

- How do we detect it when it *does* occur?
- Adding or subtracting two 32-bit numbers can yield a result that needs 33 bits to be fully expressed
- Overflow occurs in subtraction when subtract numbers
 - negative from positive and get negative result, or
 - positive from negative and get positive result
- Such result means a borrow occurred from the sign bit

Overflow Detection

- Lack of a 33rd bit means that when overflow occurs, sign bit is set with *value* of the result instead of proper sign of the result
- Since we need just one extra bit, only the sign bit can be wrong
- Overflow occurs when adding two positive numbers and sum is negative, or vice versa
- This spurious sum means a carry out occurred into the sign bit

Overflow Conditions

Operation	Operand A	Operand B	Result indicating overflow
$A + B$	≥ 0	≥ 0	< 0
$A + B$	< 0	< 0	≥ 0
$A - B$	≥ 0	< 0	< 0
$A - B$	< 0	≥ 0	≥ 0

Unsigned Numbers Overflow

- We have seen how to detect overflow for two's complement numbers in a computer
- What about overflow with unsigned integers?
 - Commonly used for memory addresses where overflows are ignored
- Computer designer must provide a way to ignore overflow in some cases and to recognize it in others

Unsigned Numbers Overflow

- MIPS solution is to have two kinds of arithmetic instructions to recognize the two choices:
- Add (**add**), add immediate (**addi**), and subtract (**sub**)
 - Cause exceptions on overflow
- Add unsigned (**addu**), add immediate unsigned (**addiu**), and subtract unsigned (**subu**)
 - Do not cause exceptions on overflow

Exception Handling

- MIPS detects overflow with an *exception* (*interrupt*)
 - Unscheduled procedure call
 - Address of overflowed instruction is saved in a register
 - Computer jumps to predefined address to invoke appropriate routine for that exception
 - Interrupted address is saved so that in some situations program can continue after corrective code is executed
- MIPS includes register called *exception program counter* (EPC) to contain the address of instruction that caused exception